

Hacker do Bem – Especialização em DevOps & DevSecOps

Sistema de Gerenciamento de Tarefas com Flask: Uma Abordagem DevSecOps

Autor: Fabiano Fernandes

Data: 12 de junho de 2026

Projeto Final de Especialização

Sumário

- 🔗 1. Introdução
 - 🔗 2. Requisitos
 - 🔗 3. Arquitetura
 - 🔗 4. Containerização
 - 🔗 5. Análise de Segurança Automatizada (SAST/SCA)
 - 🔗 6. Análise Dinâmica de Segurança (DAST)
 - 🔗 7. Entrega Contínua e Ambientes de Validação (CD)
 - 🔗 8. Feedback e Monitoramento
 - 🔗 9. Pipeline CD/CD
 - 🔗 10. Conclusão
-

1. Introdução

Descrição do Projeto

Este projeto tem como objetivo a implementação e a garantia de segurança de um sistema de gerenciamento de tarefas pessoais, sob a perspectiva de um desenvolvedor individual. A proposta surge da necessidade de solucionar dificuldades cotidianas relacionadas à organização de atividades, oferecendo uma solução estruturada para o gerenciamento eficiente do tempo e das demandas diárias.

Para viabilizar sua construção, o projeto foi fundamentado nos princípios do Ciclo de Vida de Desenvolvimento de Software (SDLC), integrando práticas modernas de DevOps e DevSecOps. Essa abordagem possibilitou a automação de fluxos de trabalho, a containerização da aplicação por meio do Docker e a incorporação de mecanismos de segurança desde as fases iniciais do desenvolvimento, garantindo maior confiabilidade e consistência ao sistema.

Escopo

O presente projeto tem como escopo o desenvolvimento de um Produto Mínimo Viável (PMV) de um sistema web para gerenciamento de tarefas pessoais. A aplicação

foi projetada para ser leve e funcional, com foco na experiência individual do usuário, garantindo isolamento de dados em um ambiente multiusuário.

O sistema contempla as seguintes funcionalidades e características:

- Cadastro e autenticação de usuários
 - Criação e exclusão de tarefas
 - Monitoramento e observabilidade da aplicação
 - Implementação de pipeline automatizado de Integração e Entrega Contínua (CI/CD)
-

2. Levantamento de Requisitos

Requisitos Funcionais

Os requisitos funcionais descrevem as funcionalidades que o sistema deve oferecer, ou seja, as ações e comportamentos esperados a partir da interação do usuário. Autenticação de usuários.

- RF01 – Autenticação de Usuário
 - O sistema deve permitir que usuários realizem cadastro, login e logout de forma segura, utilizando credenciais individuais.
- RF02 – Criação de Tarefas
 - O usuário autenticado deve ser capaz de criar novas tarefas, informando obrigatoriamente um título.
- RF03 – Visualização de Tarefas (Listagem)
 - O sistema deve permitir que o usuário visualize exclusivamente as tarefas associadas à sua conta.
- RF04 – Exclusão de Tarefas
 - O usuário deve ter a capacidade de excluir tarefas previamente cadastradas em seu painel.

Requisitos Não Funcionais

Os requisitos não funcionais definem restrições, atributos de qualidade e características técnicas que o sistema deve atender, não estando diretamente relacionados às funcionalidades de negócio, mas sim ao seu desempenho, segurança e operação.

Segurança (Alinhado a DevSecOps)

- RNF01 – Isolamento de Dados (Princípio do Menor Privilégio)
 - O sistema deve garantir que usuários autenticados não possam acessar, modificar ou excluir tarefas pertencentes a outros usuários, prevenindo vulnerabilidades do tipo IDOR (Insecure Direct Object Reference).
- RNF02 – Criptografia de Credenciais
 - As senhas dos usuários devem ser armazenadas de forma segura, utilizando algoritmos de hash criptográfico Bcrypt.

- RNF03 – Proteção de Comunicação
 - Toda comunicação entre cliente (navegador) e servidor deve ser criptografada em trânsito por meio do protocolo HTTPS.

Infraestrutura e Portabilidade (Containers & Docker)

- RNF04 – Containerização
 - A aplicação e suas dependências, incluindo o banco de dados, devem ser empacotadas de forma isolada utilizando Docker, garantindo a portabilidade e a replicação do ambiente em diferentes máquinas por meio do uso do docker-compose.
- RNF05 – Persistência de Dados
 - Os dados do banco de dados devem ser preservados mesmo após reinicializações, paradas ou atualizações dos containers, sendo obrigatório o uso de Docker Volumes.

Ciclo de Vida e Operação (DevOps & CI/CD)

- RNF06 – Automação de Builds (CI)
 - O código-fonte deve passar por um processo automatizado de build e execução de testes a cada nova alteração (ex: git push), evitando a progressão de código com falhas no ciclo de desenvolvimento.
- RNF07 – Segurança Automatizada (SAST/SCA)
 - A pipeline de integração contínua deve incorporar ferramentas de análise estática de código (SAST) e verificação de vulnerabilidades em dependências (SCA) antes da geração da imagem final da aplicação.

Consideração de Engenharia

A separação clara entre requisitos funcionais e não funcionais possibilitou a adoção do princípio de *Security by Design*, no qual aspectos de segurança são considerados desde as fases iniciais do desenvolvimento. Dessa forma, requisitos relacionados à proteção de dados, criptografia e comunicação segura orientaram as decisões arquiteturais desde o início do projeto.

Ferramentas Utilizadas:

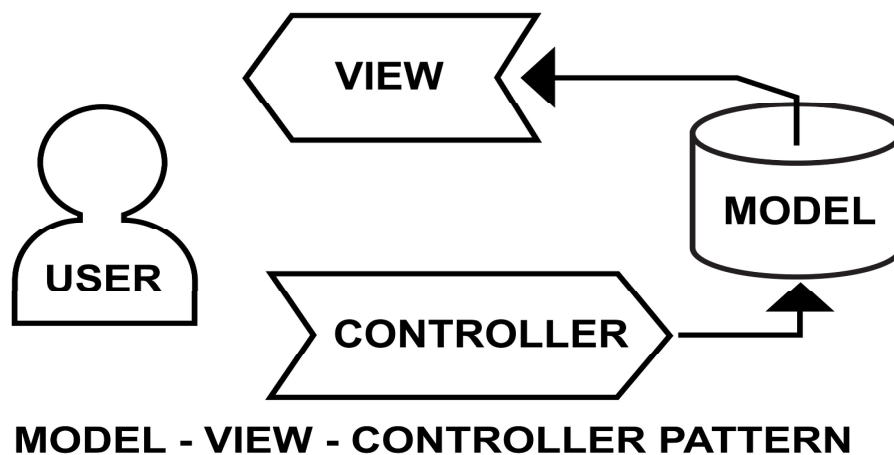
- **Desenvolvimento:** Flask (Framework web), SQLite (Gerenciamento de dados).
- **Infraestrutura:** Docker (Containerização), Git (Versionamento).
- **Segurança (DevSecOps):** * **Bandit:** Para análise estática de segurança do código-fonte Python (identificação de falhas comuns).
 - **OWASP Dependency-Check:** Para análise de composição de software (SCA), detectando vulnerabilidades conhecidas em bibliotecas de terceiros.
 - **OWASP ZAP (Zed Attack Proxy):** Para testes de segurança em tempo de execução (DAST), simulando ataques na interface da aplicação.

Resultados Alcançados: Foi estabelecida a fundação para um ambiente robusto, onde a segurança deixa de ser reativa e passa a ser integrada. A definição das ferramentas permitiu o mapeamento dos pontos de controle que serão automatizados no pipeline:

- **Análise Estática (SAST):** Uso do Bandit para auditar o código Flask a cada *build*.
 - **Análise de Dependências (SCA):** Uso do OWASP Dependency-Check para garantir que nenhuma biblioteca desatualizada comprometa o sistema.
 - **Análise Dinâmica (DAST):** Uso do OWASP ZAP para validar a superfície de ataque da aplicação em execução.
-

3. Arquitetura

Nesta etapa, foi analisada a arquitetura do sistema de gerenciamento de tarefas, com foco na organização estrutural da aplicação e na preparação do ambiente para execução em containers Docker.



A análise do código-fonte base permitiu compreender o funcionamento da aplicação e adaptar sua estrutura para um modelo mais organizado e compatível com práticas modernas de desenvolvimento.

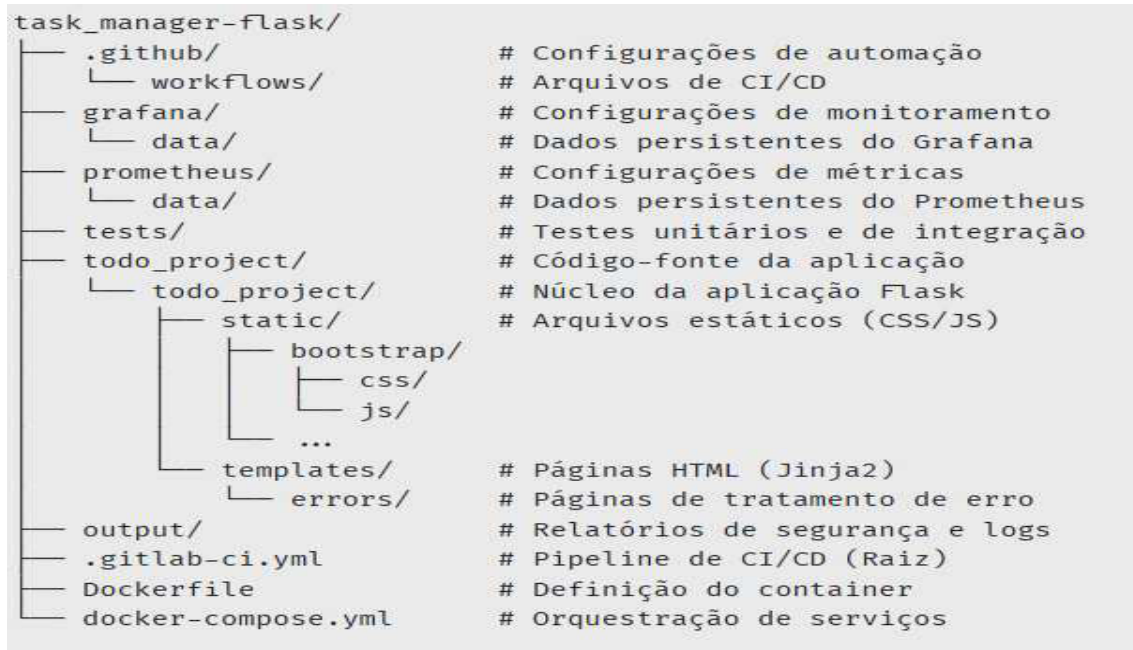
A arquitetura do sistema contempla:

- Organização modular de pastas e arquivos
- Estrutura baseada no framework Flask
- Separação entre camadas (rotas, lógica e templates)
- Modelagem simples de banco de dados para persistência das tarefas

Resultados

Foi possível estruturar corretamente a aplicação, compreendendo sua arquitetura interna e organizando os componentes de forma adequada para evolução

futura. A base do sistema foi preparada para integração com containers, facilitando sua portabilidade e execução em diferentes ambientes.



4. Containerização Padronização do Ambiente

Nesta etapa, o foco foi a encapsulação de toda a stack tecnológica da aplicação (Flask, dependências Python e persistência SQLite) em containers Docker. O objetivo central foi garantir a paridade entre os ambientes de desenvolvimento, teste e produção, eliminando o problema clássico de inconsistências de configuração e dependências ausentes (*"na minha máquina funciona"*).

Ferramentas Utilizadas:

- **Docker:** Para a definição do Dockerfile (instruções de imagem).
- **Docker Compose:** Para a orquestração multi-container e definição de redes e volumes.

Resultados Alcançados:

- **Ambiente Portável:** Criação de um ambiente que pode ser replicado integralmente com o comando `docker-compose up`.
- **Isolamento:** A aplicação e o banco de dados rodam em containers distintos, mas conectados via rede virtual, garantindo maior segurança e organização.
- **Persistência:** Implementação de volumes Docker para garantir que os dados persistam após a remoção ou reinicialização dos containers (atendendo ao requisito RNF05).

Dockerfile

```
Dockerfile > ...  
...  
1 FROM python:3.11-slim  
2  
3 WORKDIR /app  
4  
5 #ENV PYTHONPATH=/app  
6 ENV PYTHONPATH=/app:/app/todo_project  
7  
8 COPY requirements.txt .  
9 RUN pip install --no-cache-dir -r requirements.txt  
10  
11 COPY . .  
12  
13 |  
14 CMD ["python", "todo_project/run.py"]  
15
```

docker-compose.yml

```
1 services:  
  >Run Service  
2   web:  
3     build: .  
4     ports:  
5       - "5000:5000"  
6       - "9100:9100" # Expondo a porta de métricas que configuramos no run.py  
7     volumes:  
8       - ./data:/app/data  
9     environment:  
10      - PYTHONPATH=/app:/app/todo_project  
11     networks:  
12      - monitoramento_network  
13
```

5. Análise de Segurança Automatizada (SAST/SCA)

A segurança foi integrada diretamente na esteira de CI/CD, utilizando ferramentas de análise estática e de composição de software. O objetivo foi automatizar a detecção de vulnerabilidades no código-fonte e em bibliotecas de terceiros a cada novo *commit*, garantindo que falhas de segurança não cheguem aos ambientes de estágio ou produção (*Shift Left Security*).

Ferramentas Utilizadas:

- **Bandit:** Para análise estática de código (SAST), focada em identificar padrões inseguros em Python.
- **OWASP Dependency-Check:** Para análise de dependências (SCA), identificando vulnerabilidades conhecidas (CVEs) em pacotes listados no `requirements.txt`.

Resultados Alcançados:

- Implementação de estágios de segurança que validam o código antes de qualquer build de container.
- Geração de relatórios de vulnerabilidade automatizados, permitindo a correção proativa pelo desenvolvedor.

Análise do Job:

- **Bandit:** Configurado para ignorar o aviso de B104 (binding a IP 0.0.0.0), que é comum e necessário em ambientes Docker, enquanto mantém o foco em falhas de severidade alta e confiança alta.
- **pip-audit & OWASP:** Garantem que bibliotecas desatualizadas ou com CVEs (Common Vulnerabilities and Exposures) sejam identificadas antes do deploy.
- **Política de Falha:** O parâmetro `--failOnCVSS 7` estabelece um *gate* de segurança que interrompe o processo caso qualquer vulnerabilidade de severidade alta (ou crítica) seja detectada.

```

87 - name: Instalar dependências de segurança
88   run: |
89     pip install --upgrade pip
90     pip install bandit pip-audit
91
92 - name: 🛡️ Bandit (SAST)
93   run: |
94     bandit -r . -ll -ii -s B104
95
96 - name: 🛡️ pip-audit (dependências)
97   run: |
98     pip-audit
99
100 - name: 🛡️ OWASP Dependency Check
101   uses: dependency-check/Dependency-Check_Action@main
102   with:
103     project: "task_manager"
104     path: "."
105     format: "HTML"
106     args: >
107       --failOnCVSS 7
108       --enableRetired
109
110 - name: Upload relatório OWASP
111   uses: actions/upload-artifact@v4
112   with:
113     name: dependency-check-report
114     path: reports/

```

6. Análise Dinâmica de Segurança (DAST)

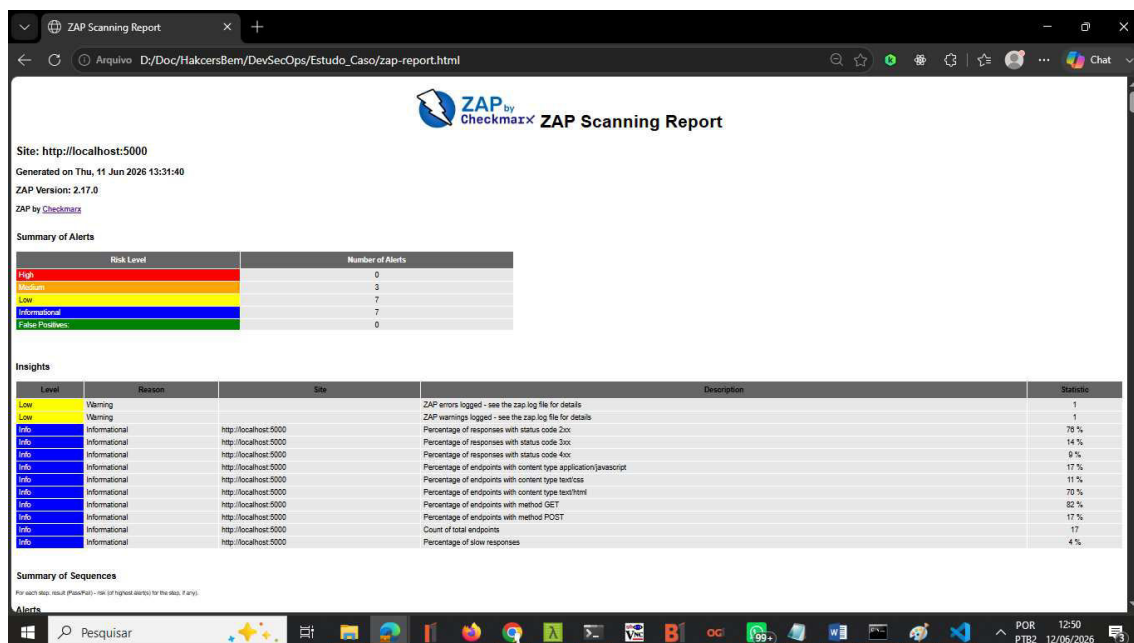
Diferente da análise estática (que analisa o código), a Análise Dinâmica (DAST) foi realizada contra a aplicação em execução (em ambiente de container). Utilizamos o **OWASP ZAP** para simular tentativas de invasão e identificar vulnerabilidades que só aparecem quando o sistema está ativo, como configurações incorretas de cabeçalhos HTTP, exposição de diretórios ou falhas de autenticação.

Ferramentas Utilizadas:

- **OWASP ZAP (Zed Attack Proxy):** Ferramenta de teste de penetração automatizada.
- **Docker:** Para execução da imagem oficial do ZAP em modo *headless*.

Resultados Alcançados:

- Execução de varredura automatizada contra o container da aplicação (web).
- Geração de relatório de vulnerabilidades em tempo real.
- Validação da resiliência da aplicação frente a ataques comuns (ex: XSS, SQL Injection básico).



7. Entrega Contínua e Ambientes de Validação (CD)

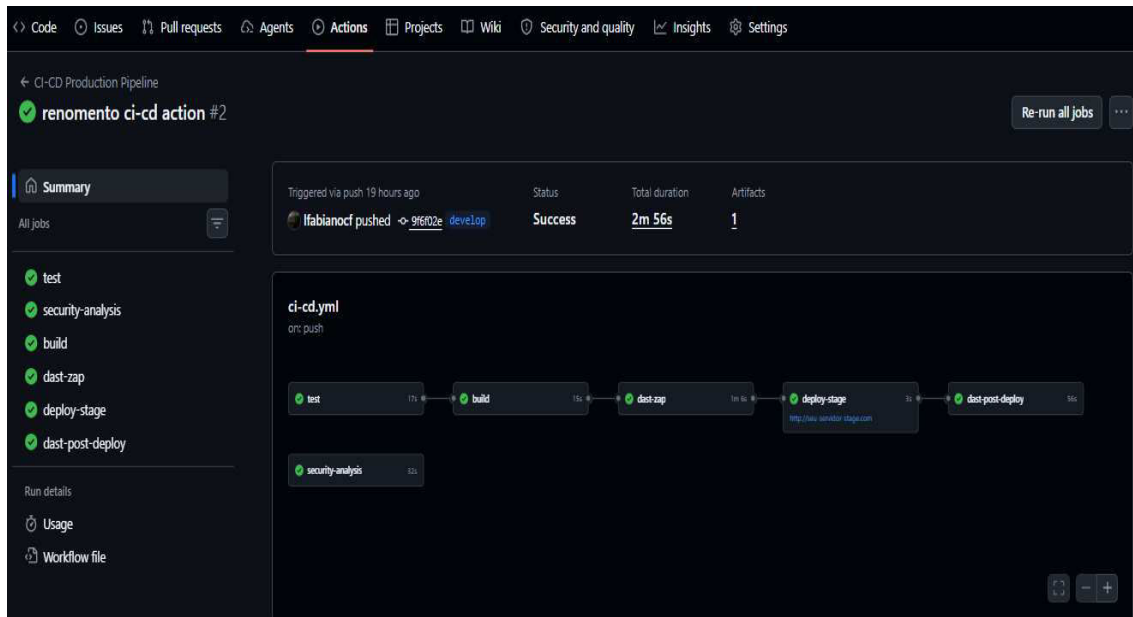
A fase de Entrega Contínua (CD) foi configurada para automatizar o *deploy* em ambientes de homologação (*staging*) e revisão. Esta etapa garante que, após a integração (CI), o código seja exposto em um ambiente idêntico à produção para validações finais, incluindo a execução de testes DAST (OWASP ZAP) em tempo de execução para assegurar que a aplicação, quando implantada, permanece segura.

Fluxo de Trabalho de Entrega:

1. **Ambiente de Review:** Criação automática de instâncias temporárias para avaliação manual em cada *Merge Request*.
2. **Deploy em Estágio:** Implantação automática no ambiente de *staging* após o sucesso nos testes de unidade e SAST.
3. **DAST Pós-Deploy:** Execução do scan do OWASP ZAP contra o ambiente de *staging* para validar a segurança da aplicação publicada.

Resultados Alcançados:

- Automatização do ciclo de *feedback* rápido entre desenvolvimento e operação.
- Garantia de que a aplicação em estágio é submetida a testes de estresse e segurança antes da aprovação final.
- Redução de riscos de *deploy* através do uso de ambientes de *review*.
- O job *deploy-staging* depende (needs) da conclusão bem-sucedida da *security-analysis* e *dast-scan*.



8. Feedback e Monitoramento

Nesta etapa, foi implementado o monitoramento contínuo da aplicação após sua implantação, com o objetivo de garantir visibilidade sobre o comportamento do sistema e detectar possíveis anomalias em tempo real. A abordagem adotada segue os princípios de observabilidade, permitindo a coleta, análise e visualização de métricas e eventos da aplicação em execução.

Implementação

O monitoramento foi realizado por meio da integração de ferramentas especializadas em coleta de métricas e visualização de dados:

- **Coleta de Métricas com Prometheus:**
 - O Prometheus foi configurado para coletar métricas da aplicação, incluindo desempenho, uso de recursos e disponibilidade dos serviços.
- **Visualização com Grafana:**

- O Grafana foi utilizado para criar dashboards interativos, permitindo a análise visual das métricas coletadas e facilitando a identificação de comportamentos anômalos.
- **Monitoramento de Logs e Anomalias:**
 - Embora o foco principal tenha sido métricas, também foi considerada a importância da análise de logs para identificação de eventos suspeitos, como tentativas de acesso indevido ou ataques de força bruta.

Essas ferramentas foram integradas ao ambiente containerizado, permitindo monitoramento contínuo após o deploy da aplicação.

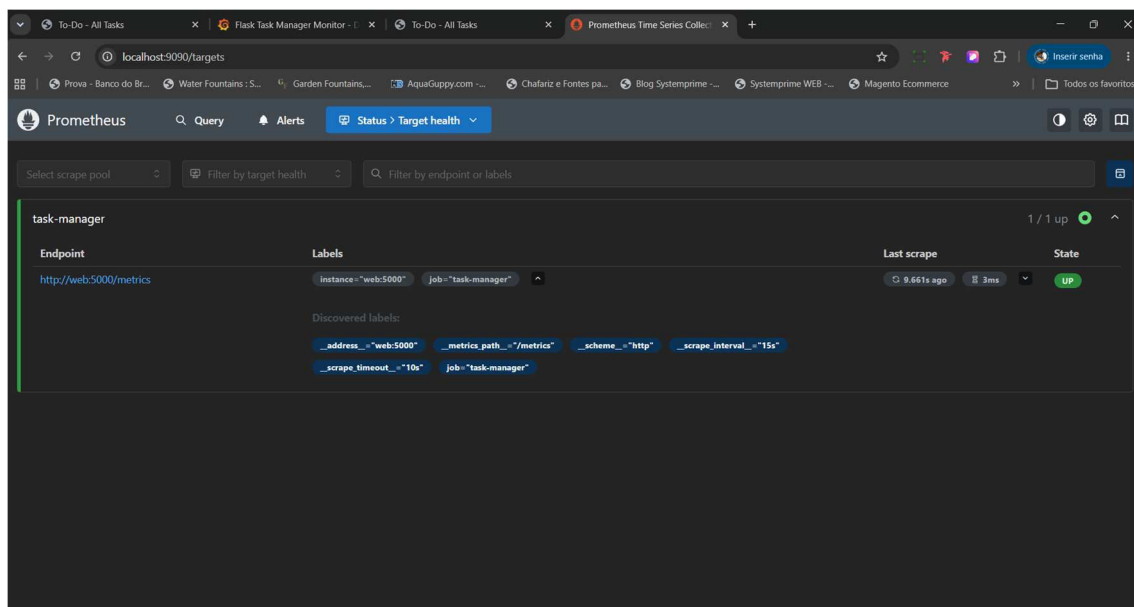
Ferramentas Utilizadas

- Prometheus (coleta de métricas)
- Grafana (visualização e dashboards)
- Docker / Docker Compose (integração dos serviços)

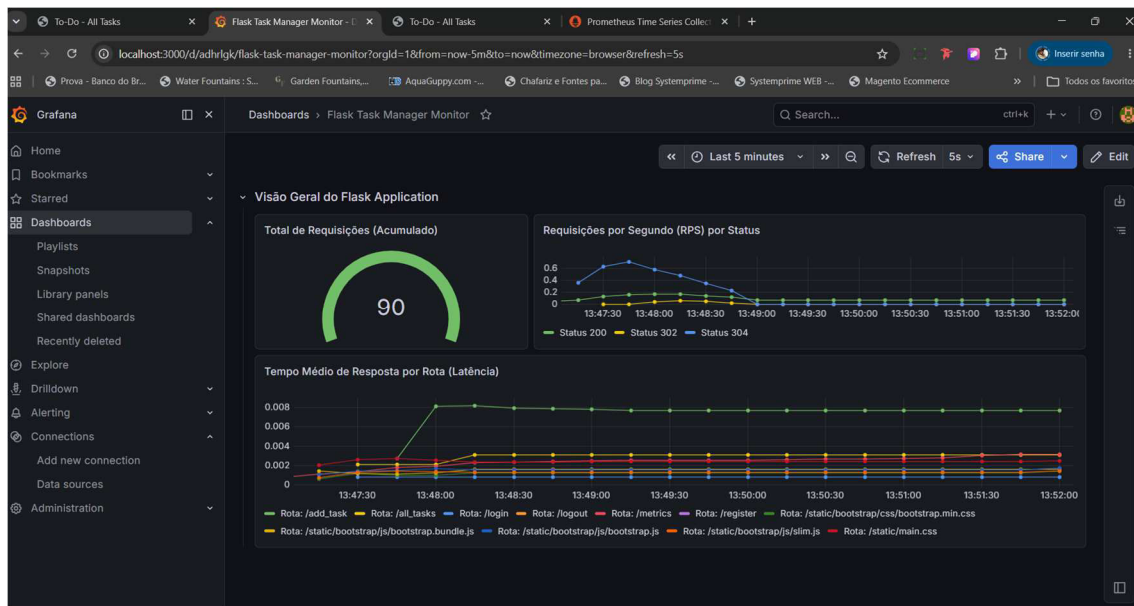
Resultados

A implementação do monitoramento permitiu obter maior visibilidade sobre o funcionamento da aplicação em tempo real. Foi possível acompanhar métricas de desempenho e identificar comportamentos anormais, contribuindo para a estabilidade e confiabilidade do sistema.

Prometheus



Grafana



9. CI/CD – Github Actions

name: CI-CD Production Pipeline

concurrency:

group: \${{ github.workflow }}-\${{ github.ref }}

cancel-in-progress: true

on:

push:

branches: ["main", "develop"]

pull_request:

branches: ["main", "develop"]

workflow_dispatch:

jobs:

=====

1. TESTS

=====

test:

runs-on: ubuntu-latest

steps:

- name: Checkout do código

uses: actions/checkout@v4

- name: Set up Python

uses: actions/setup-python@v5

with:

python-version: "3.11"

Configuração correta do Cache para acelerar os próximos deploys

- name: Cache pip

uses: actions/cache@v4

with:

path: ~/.cache/pip

key: \${{ runner.os }}-pip-\${{ hashFiles('**/requirements.txt') }}

```

    restore-keys: |

    ${{ runner.os }}-pip-

    # Instalação das dependências fora do
    bloco de configuração do cache

    - name: Instalar dependências

    run: |

        pip install -r requirements.txt

        pip install pytest

    - name: Run tests

    env:

        PYTHONPATH: ${{ github.workspace
        }}/todo_project

    run: |

        python -m pytest tests/ -v

# =====

# 🛠️ 2. BUILD (DOCKER)

# =====

build:

    needs: test # O build só começa se o job
    'test' terminar com sucesso

    runs-on: ubuntu-latest

steps:

    - name: Checkout do código

      uses: actions/checkout@v4

    - name: Build Docker image

      run: |

        docker build -t task-manager:${{
        github.sha }}.

```

```

# =====

# 🛡️ 3. SECURITY ANALYSIS

# =====

security-analysis:

    runs-on: ubuntu-latest

steps:

    - name: Checkout do código

      uses: actions/checkout@v4

    - name: Set up Python

      uses: actions/setup-python@v5

      with:

        python-version: "3.11"

    - name: Cache pip

      uses: actions/cache@v4

      with:

        path: ~/.cache/pip

        key: ${{ runner.os }}-pip-${{
        hashFiles('**/requirements.txt') }}

    - name: Instalar dependências de
    segurança

      run: |

        pip install --upgrade pip

        pip install bandit pip-audit

    - name: 🛡️ Bandit (SAST)

      run: |

        bandit -r . -ll -ii -s B104

```

```

- name: 🐛 pip-audit (dependências)

  run: |

    pip-audit


- name: 🐛 OWASP Dependency Check

  uses: dependency-check/Dependency-Check_Action@main

  with:

    project: "task_manager"

    path: "."

    format: "HTML"

    args: >

      --failOnCVSS 7

      --enableRetired


- name: Upload relatório OWASP

  uses: actions/upload-artifact@v4

  with:

    name: dependency-check-report

    path: reports/


# =====

# 💚 4. DAST - OWASP ZAP

# =====

dast-zap:

  needs: build

  runs-on: ubuntu-latest


steps:

- name: Checkout do código

  uses: actions/checkout@v4

```

```

# 1. Sobe o app na MESMA máquina
onde o ZAP vai rodar

```

```

- name: Subir aplicação com Docker
Compose

```

```

run: |

  docker compose up -d --build

```

```

# 2. Aguarda o Flask inicializar
completamente (Healthcheck interno)

```

```

- name: Healthcheck Flask

```

```

run: |

  echo "Waiting for app..."

  for i in {1..30}; do

    if curl -s http://localhost:5000 >
/dev/null; then

      echo "App is UP on localhost:5000!"

      exit 0

    fi

    echo "attempt $i..."

    sleep 2

  done

  echo "App failed to start"

  docker compose logs web

  exit 1

```

```

# 3. Agora o ZAP consegue achar o
localhost:5000 com permissão de escrita!

```

```

- name: OWASP ZAP scan

```

```

run: |

```

```

# ★ ADICIONADO: --user root para o
Linux permitir a gravação do relatório

```

```
# ★ AJUSTADO: -r zap-report.html (o
ZAP joga direto no seu $(pwd) através do
volume)
```

```
docker run --rm --user root --network
host -v $(pwd):/zap/wrk/:rw \
```

```
ghcr.io/zaproxy/zaproxy:stable zap-
baseline.py \
```

```
-t http://localhost:5000 \
```

```
-r zap-report.html \
```

```
-J zap-report.json \
```

```
-X zap-report.xml || true
```

```
# 4. Remove os containers para limpar a
máquina após o teste
```

```
- name: Stop containers
```

```
if: always()
```

```
run: docker compose down
```

```
# 5. Salva o relatório gerado para você
baixar na aba Actions
```

```
- name: Upload report
```

```
uses: actions/upload-artifact@v4
```

```
if: always()
```

```
with:
```

```
name: zap-reports-triade
```

```
path: ./zap-report.* # ★ GARANTIDO:
Busca o arquivo na raiz do workspace
```

```
# ... (mantendo seus jobs anteriores: test,
build, security-analysis, dast-zap) ...
```

```
# =====
```

```
# 🚀 5. DEPLOY EM ESTÁGIO (CD)
```

```
# =====
```

```
deploy-stage:
```

```
needs: [dast-zap]
```

```
runs-on: ubuntu-latest
```

```
environment:
```

```
name: stage
```

```
url: http://seu-servidor-stage.com
```

```
if: github.ref == 'refs/heads/develop'
```

```
steps:
```

```
- name: Deploy Staging
```

```
run: |
```

```
echo "Simulando deploy no ambiente
de estágio..."
```

```
# Adicione aqui o seu comando de
deploy, ex: scp ou ssh para seu servidor
```

```
# ssh user@seu-servidor 'cd /app &&
docker compose pull && docker compose
up -d'
```

```
# =====
```

```
# ☹️ 6. DAST PÓS-DEPLOY (STAGING)
```

```
# =====
```

```
dast-post-deploy:
```

```
needs: [deploy-stage]
```

```
runs-on: ubuntu-latest
```

```
steps:
```

```
- name: OWASP ZAP Scan em Staging
```

```
run: |
```

```
docker run --rm -v
$(pwd):/zap/wrk/:rw \
```

```
ghcr.io/zaproxy/zaproxy:stable zap-
baseline.py \
```

```
-t http://seu-servidor-stage.com \
```

```
-r zap-stage-report.html || true
```

- name: Upload relatório de Staging
uses: actions/upload-artifact@v4
with:
 name: zap-report-staging
 path: ./zap-stage-report.html

10. Conclusão

A execução deste projeto proporcionou uma vivência prática e completa do ciclo de vida de uma aplicação moderna, desde a concepção orientada à segurança (Security by Design) até a implementação de um ambiente robusto de monitoramento e observabilidade.

A evolução de uma aplicação monolítica simples para um sistema containerizado, integrado a um pipeline de CI/CD, evidenciou que a automação vai além de um recurso técnico, tornando-se um elemento essencial para garantir confiabilidade, escalabilidade e consistência no desenvolvimento de software.

A integração de ferramentas de análise estática (SAST), análise de dependências (SCA) e análise dinâmica (DAST) demonstrou, na prática, a importância do conceito de segurança contínua (Shift Left), permitindo a identificação precoce de vulnerabilidades ao longo do ciclo de desenvolvimento.

Além disso, a adoção de ferramentas de monitoramento como Prometheus e Grafana possibilitou a criação de um ciclo de feedback contínuo, transformando métricas operacionais em informações relevantes para a manutenção, segurança e evolução do sistema.

Dessa forma, o sistema desenvolvido não apenas atende aos requisitos funcionais propostos, mas também apresenta uma arquitetura resiliente, observável, auditável e alinhada às boas práticas modernas de DevSecOps, estando preparado para cenários reais de produção.